

Fundamentals: Markov Decision Process

Kianté Brantley
CS 1840 Fall 2026

Course Overview

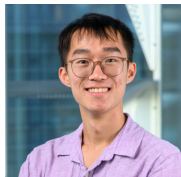
Instructor: Kianté Brantley

TFs: Alex Cai, Philip Nielsen, Egil Rolstad

Course Overview

Instructor: Kianté Brantley

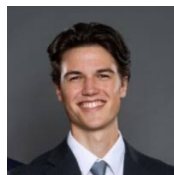
TFs: Alex Cai,



Philip Nielsen,



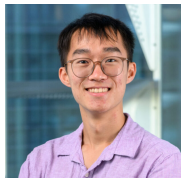
Egil Rolstad



Course Overview

Instructor: Kianté Brantley

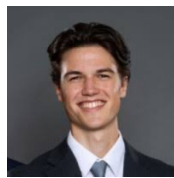
TFs: Alex Cai,



Philip Nielsen,



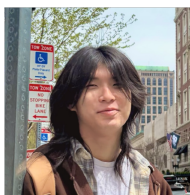
Egil Rolstad



CAs: Emmanuel Rassou,



Justin Li,



Ishaan Sinha



Announcements

Office Hours

website: <https://harvard-cs-1840.github.io/2026/>

Type	Day	Start	End	Staff	Campus	Location
Office Hours	Monday	3:45 PM	5:30 PM	Kianté	Allston	SEC 5.402
Office Hours	Monday	8:00 PM	9:30 PM	Justin	Cambridge	Mather Pset Nights
Office Hours	Thursday	2:15 PM	4:00 PM	Emmanuel	Allston	SEC 2.122/2.123
Office Hours	Thursday	4:30 PM	6:15 PM	Alex	Cambridge	Cruft Lab 309
Office Hours	Tuesday	3:45 PM	5:30 PM	Egil	Allston	SEC 2.122/2.123
Office Hours	Tuesday	7:00 PM	8:45 PM	Philip	Cambridge	Pierce Hall 301
Office Hours	Wednesday	7:00 PM	8:45 PM	Ishaan	Cambridge	Winthrop CS Nights
Section	Monday	2:15 PM	3:30 PM	Alex	Allston	SEC 1.412
Section	Thursday	12:45 PM	2:00 PM	Emmanuel	Allston	SEC 1.412
Section	Tuesday	11:15 AM	12:30 PM	Egil	Allston	SEC 1.412
Section	Tuesday	4:30 PM	5:45 PM	Philip	Cambridge	Pierce Hall 213 (Brooks Room)
Section	Wednesday	4:30 PM	5:45 PM	Ishaan & Justin	Cambridge	Maxwell Dworkin G135

Announcements

Homework 1 released 9/11 (**this Friday**)

Homework 0 due 9/17 (**next Friday**)

If any part of Homework 0 feels challenging or unfamiliar,
please reach out to the TFs/CAs for help.

Reading Materials

Three Books:

1) Reinforcement Learning: Theory and Algorithms

by Alekh Agarwal, Kianté Brantley, Nan Jiang, Sham Kakade, and Wen Sun.

This is an advanced RL theory book and is mostly out of scope for this class.

Please let us know if you find any typos or mistakes in the book

2) An Introduction to Reinforcement Learning

by Alex Cai

3) Reinforcement Learning: An Introduction

by Richard Sutton and Andrew Barto.

This is the classic text in reinforcement learning.

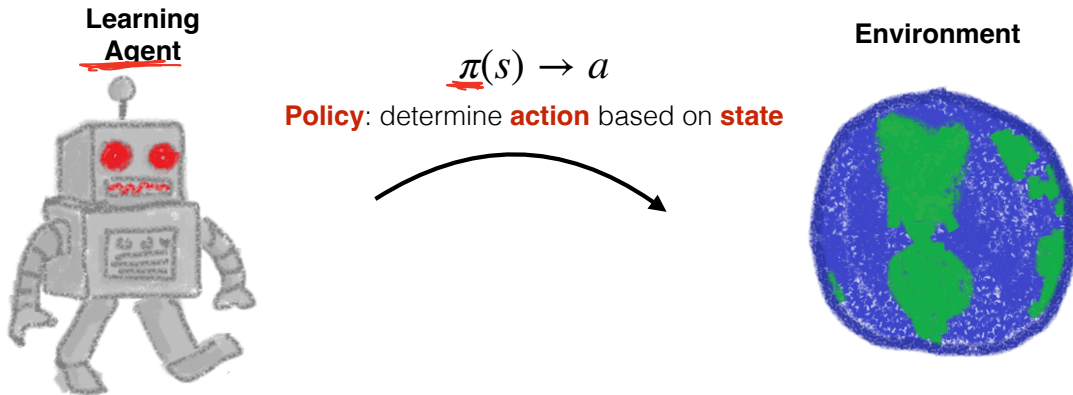
Questions?

Outline:

1. Definitions of Markov Decision Process
2. Value functions (V and Q functions)
3. Bellman equations

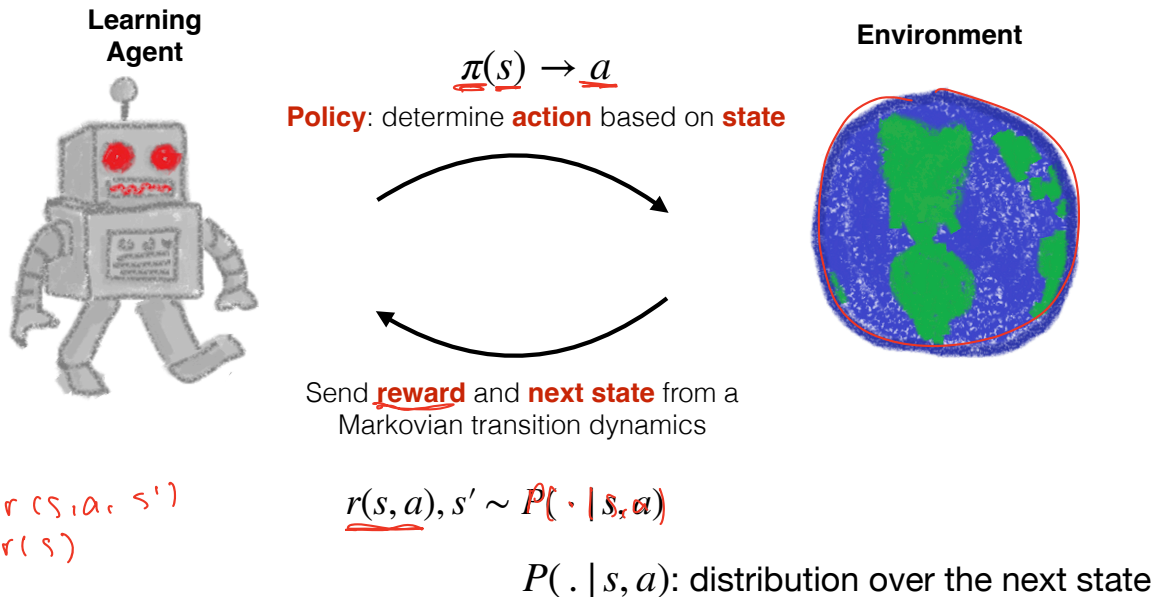
The Mathematical framework:

Infinite horizon Markov Decision Process



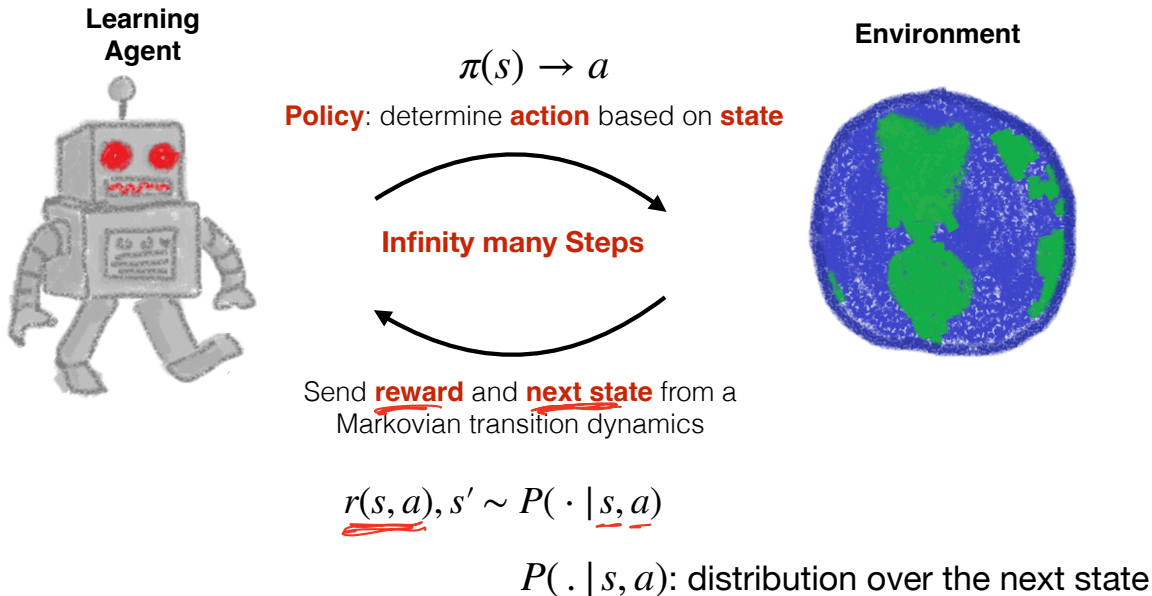
The Mathematical framework:

Infinite horizon Markov Decision Process



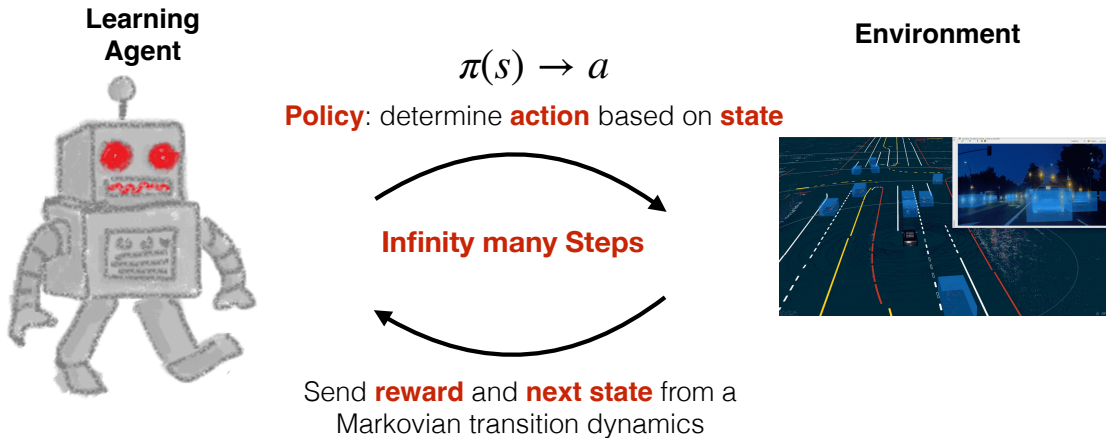
The Mathematical framework:

Infinite horizon Markov Decision Process



The Mathematical framework:

Infinite horizon Markov Decision Process

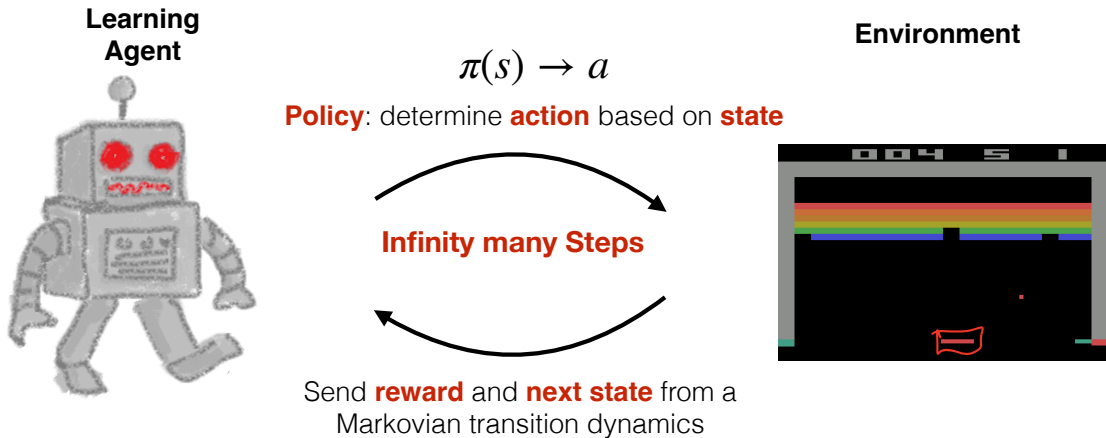


$$r(s, a), s' \sim P(\cdot | s, a)$$

$P(\cdot | s, a)$: distribution over the next state

The Mathematical framework:

Infinite horizon Markov Decision Process

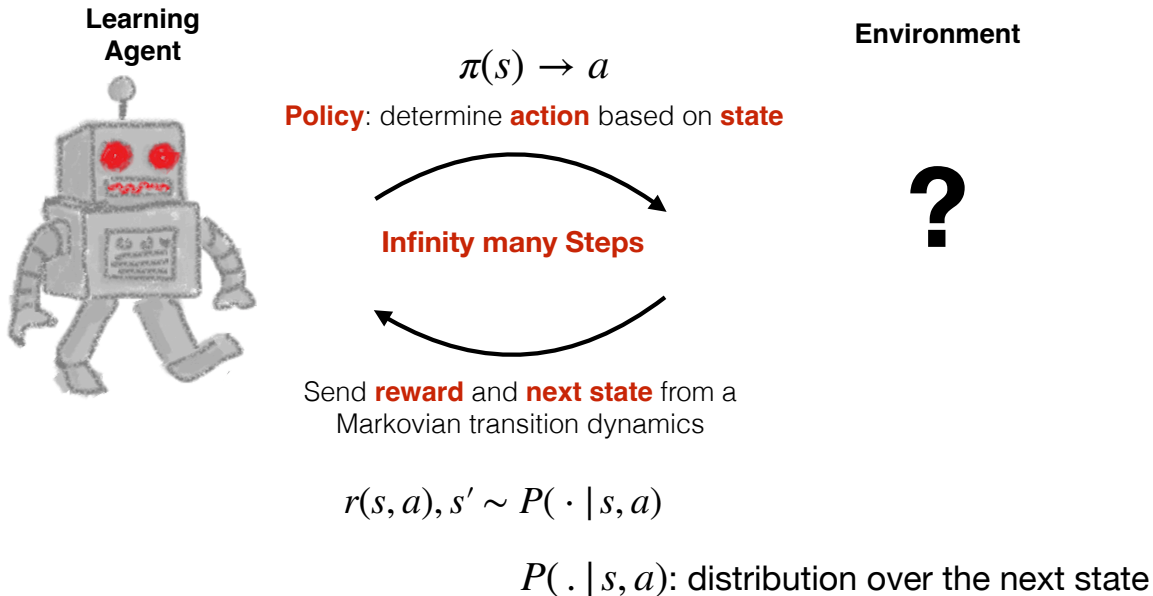


$$\underline{r}(s, a), s' \sim P(\cdot | s, a)$$

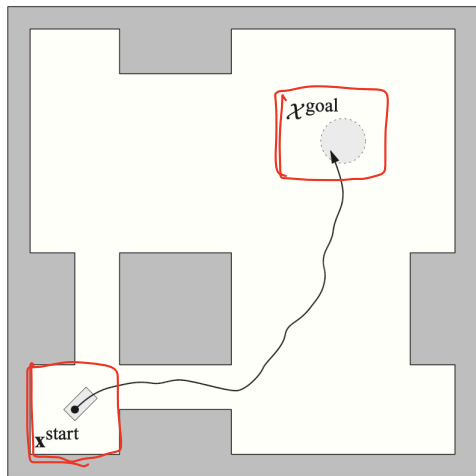
$P(\cdot | s, a)$: distribution over the next state

The Mathematical framework:

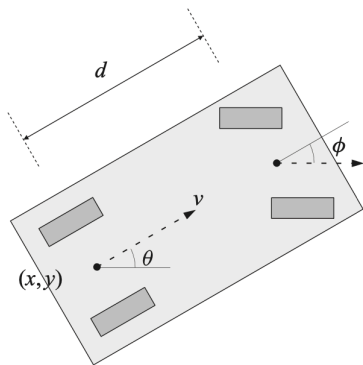
Infinite horizon Markov Decision Process



Example: 2-D simple car navigation

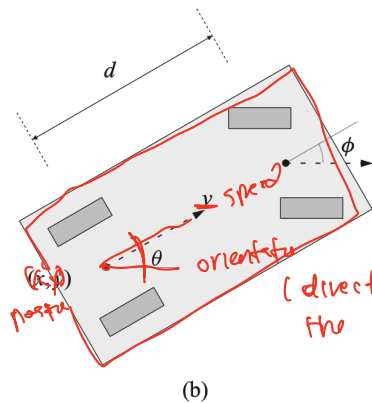
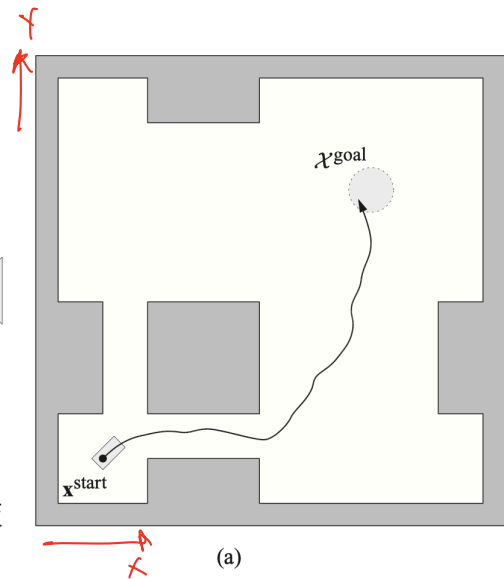


(a) Environment



(b) Agent: Car

Example: 2-D simple car navigation



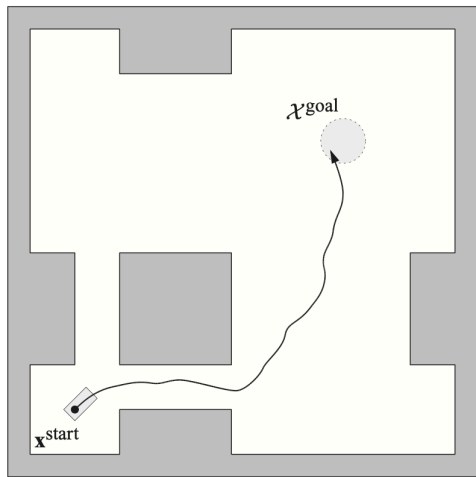
State: $s = [x, y, \theta, v]^T \in \mathbb{R}^4$

Handwritten notes in red:

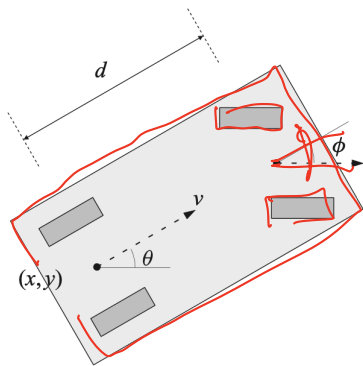
- x, y : position of the car
- θ : orientation
- v : speed

(direction in which the car is moving)

Example: 2-D simple car navigation



(a)



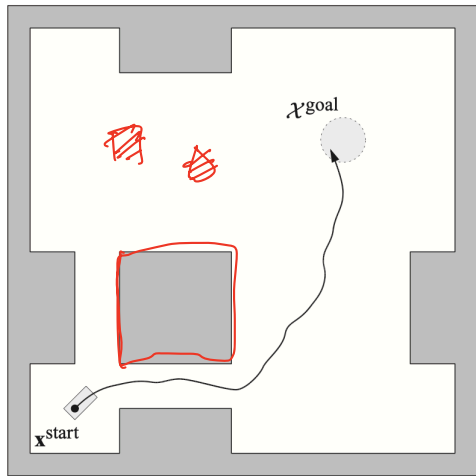
(b)

$$s = [x, y, \theta, v]^T \in \mathbb{R}^4$$

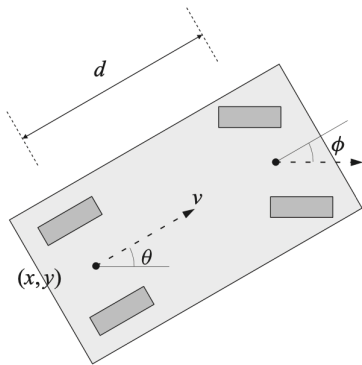
action: $a = [\alpha, \phi]^T \in \mathbb{R}^2$

acceleration
steering angle

Example: 2-D simple car navigation



(a)



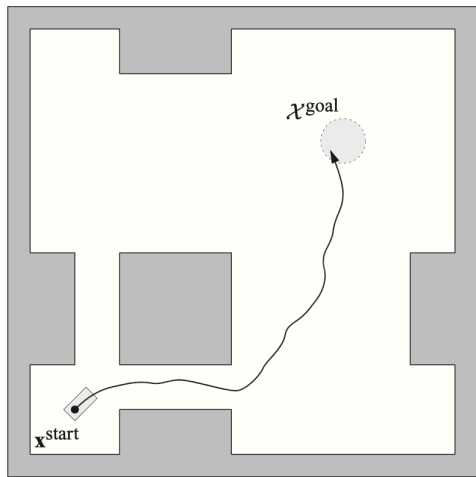
(b)

$$s = [x, y, \theta, v]^T \in \mathbb{R}^4$$

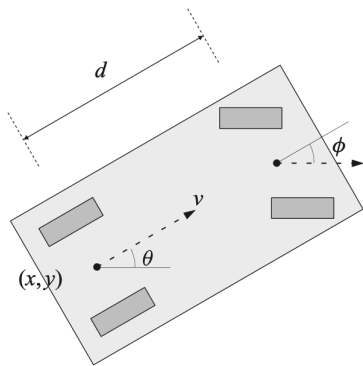
$$a = [\alpha, \phi]^T \in \mathbb{R}^2$$

$$\underline{r}(s, \underline{a}) = \begin{cases} \underline{100} & (x, y) \in \mathcal{X}_{\text{goal}} \\ \underline{-1} & \text{hit obstacles} \\ 0 & \text{else} \end{cases}$$

Example: 2-D simple car navigation



(a)



(b)

$$s = [x, y, \theta, v]^T \in \mathbb{R}^4$$

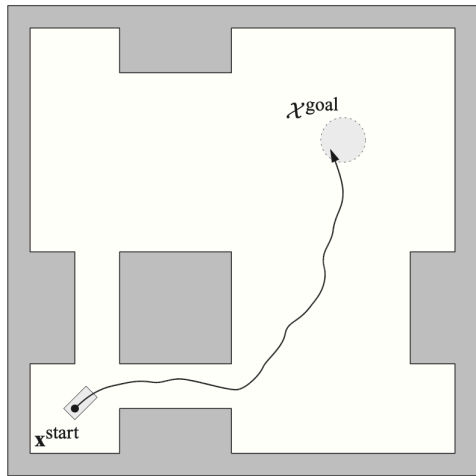
$$a = [\alpha, \phi]^T \in \mathbb{R}^2$$

$$r(s, a) = \begin{cases} 100 & (x, y) \in \mathcal{X}_{\text{goal}} \\ -1 & \text{hit obstacles} \\ 0 & \text{else} \end{cases}$$

$$s' = f(s, a) + \epsilon, \text{ where } \epsilon \sim \mathcal{N}(0, I)$$

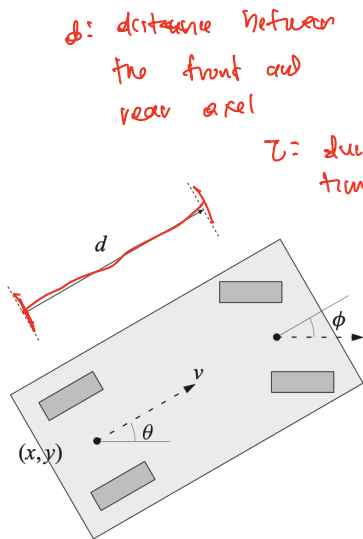
dynamic models

Example: 2-D simple car navigation



(a)

$$s' = \begin{bmatrix} x + \epsilon \\ y + \epsilon \\ \theta + \epsilon \\ v + \epsilon \end{bmatrix} = f(s, a) + \epsilon$$



(b)

d : distance between
the front and
rear axle

τ : duration before
turn

$$s = [x, y, \theta, v]^T \in \mathbb{R}^4$$

$$a = [\alpha, \phi]^T \in \mathbb{R}^2$$

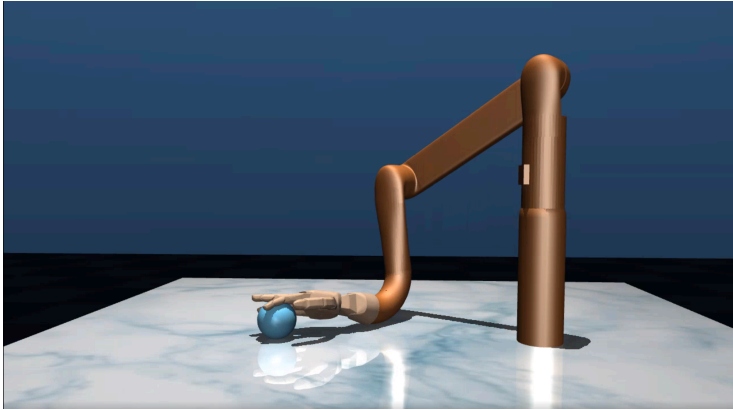
$$r(s, a) = \begin{cases} 100 & (x, y) \in \mathcal{X}_{\text{goal}} \\ -1 & \text{hit obstacles} \\ 0 & \text{else} \end{cases}$$

$$s' = f(s, a) + \epsilon, \text{ where } \epsilon \sim \mathcal{N}(0, I)$$

$$f(s, a) = \begin{bmatrix} x' \\ y' \\ \theta' \\ v' \end{bmatrix} = \begin{bmatrix} x + \tau v \cos \theta \\ y + \tau v \sin \theta \\ \theta + \tau v \tan(\phi) / d \\ v + \tau \alpha \end{bmatrix}$$

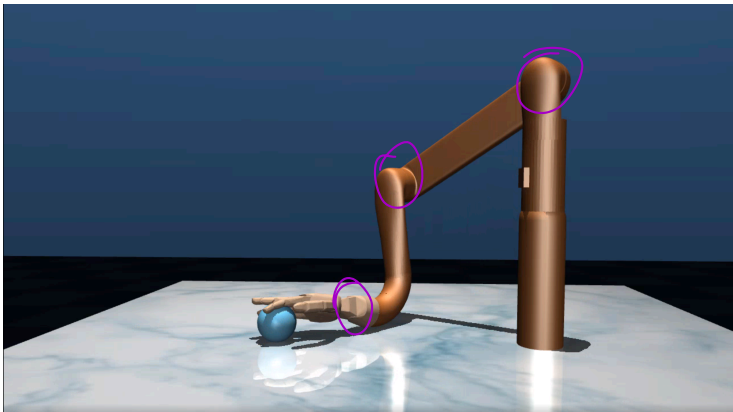
Example: Robotic Pick-and-Place

robot hand needs to pick the ball and hold it in a goal (x,y,z) position



Example: Robotic Pick-and-Place

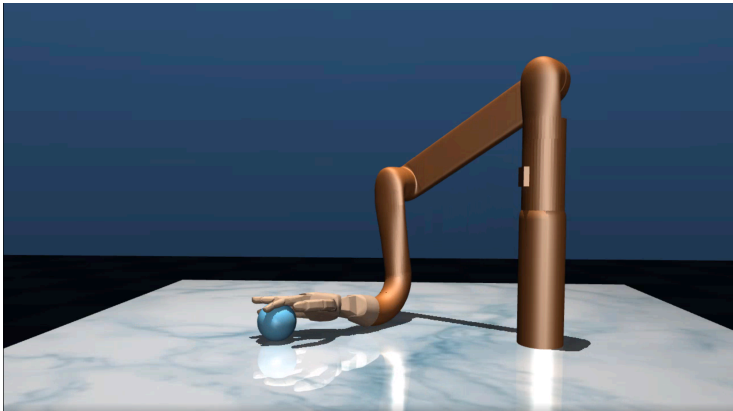
robot hand needs to pick the ball and hold it in a goal (x,y,z) position



State s : robot configuration (e.g., joint angles)
and the ball's position

Example: Robotic Pick-and-Place

robot hand needs to pick the ball and hold it in a goal (x,y,z) position

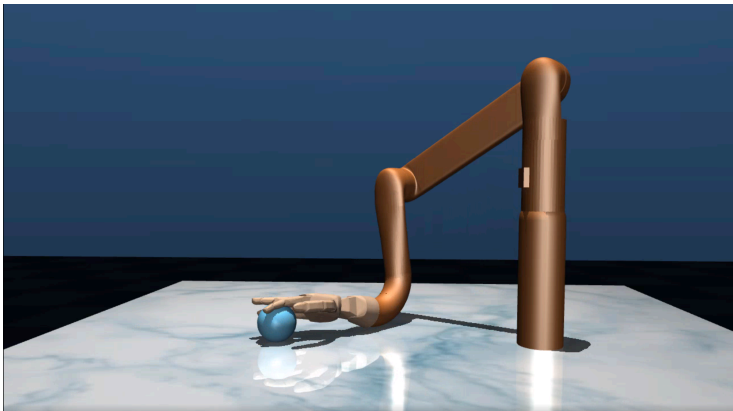


State s : robot configuration (e.g., joint angles)
and the ball's position

Action a : Torque on joints in arm & fingers

Example: Robotic Pick-and-Place

robot hand needs to pick the ball and hold it in a goal (x,y,z) position



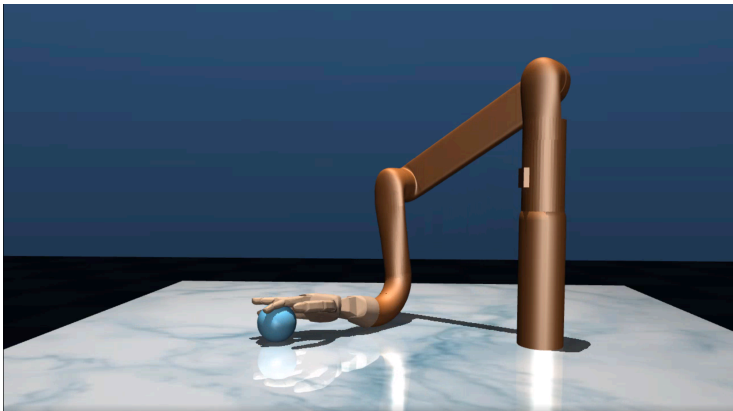
State s : robot configuration (e.g., joint angles)
and the ball's position

Action a : Torque on joints in arm & fingers

Transition $s' \sim P(\cdot | s, a)$: physics + some noise

Example: Robotic Pick-and-Place

robot hand needs to pick the ball and hold it in a goal (x,y,z) position



cost = -reward

State s : robot configuration (e.g., joint angles) and the ball's position

Action a : Torque on joints in arm & fingers

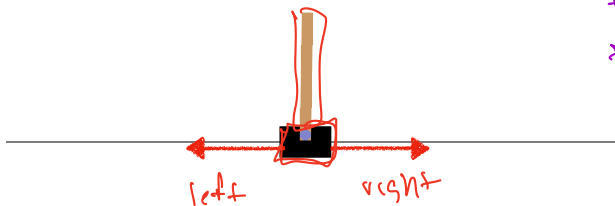
Transition $s' \sim P(\cdot | s, a)$: physics + some noise

Cost $c(s, a)$: torque magnitude + dist to goal

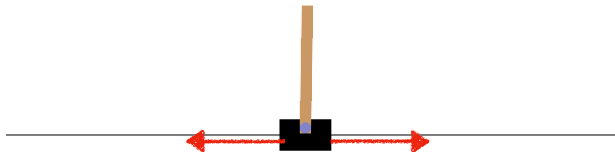
Example: OpenAI Gym demonstrations

Cart pole

Goal: Train an agent
to keep the pole
from falling over

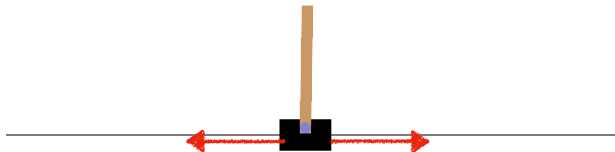


Example: OpenAI Gym demonstrations



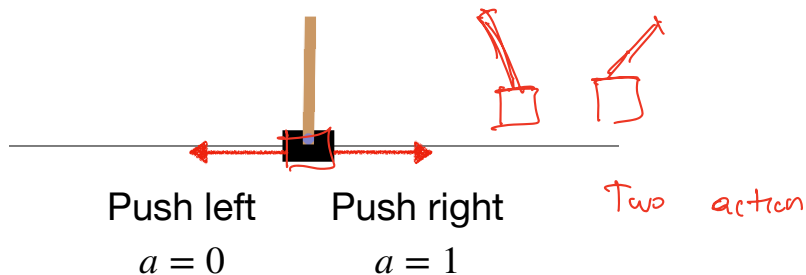
Example: OpenAI Gym demonstrations

State = [cart pos, cart velocity,
pole angle, pole angular velocity]



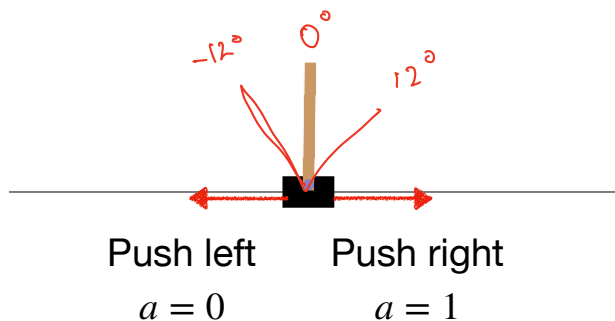
Example: OpenAI Gym demonstrations

State = [cart pos, cart velocity,
pole angle, pole angular velocity]



Example: OpenAI Gym demonstrations

State = [cart pos, cart velocity,
pole angle, pole angular velocity]



$$r(s, a) = \begin{cases} 1 & \text{pole angle} \in [-12^\circ, 12^\circ], \\ 0 & \text{else} \end{cases}$$

Policy

set S state space

action space

$$\pi: \mathcal{S} \rightarrow \Delta(\mathcal{A})$$

distribution

A mapping from state to action (what action should I take if I'm in this state...)

$$a \sim \pi(\cdot | s)$$

$$\pi(\cdot | s) \in \Pi(\mathcal{A})$$



Action space

$$\mathcal{A} = \{ \text{left}, \text{right} \}$$

Set all possible distributions

$$(0.3, 0.7) \in \Pi(\mathcal{A})$$

$$(0.2, 0.8) \in \Pi(\mathcal{A})$$

$\vdots$

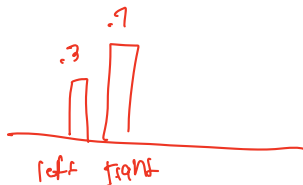
Policy

$$\mathcal{S} \rightarrow \Delta(\mathcal{A})$$

A mapping from state to action (what action should I take if I'm in this state...)

$$a \sim \pi(\cdot | s)$$

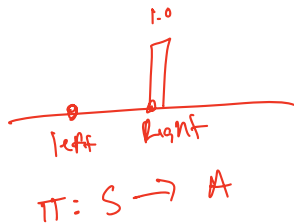
conditional
distribution over all
actions



Policy

$$\mathcal{S} \rightarrow \Delta(\mathcal{A})$$

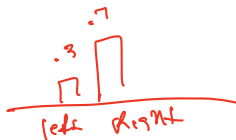
A mapping from state to action (what action should I take if I'm in this state...)



$$a \sim \pi(\cdot | s)$$

conditional
distribution over all
actions

Deterministic vs stochastic?



Policy

$$\mathcal{S} \rightarrow \Delta(\mathcal{A})$$

A mapping from state to action (what action should I take if I'm in this state...)

$$a \sim \pi(\cdot | s)$$

conditional
distribution over all
actions

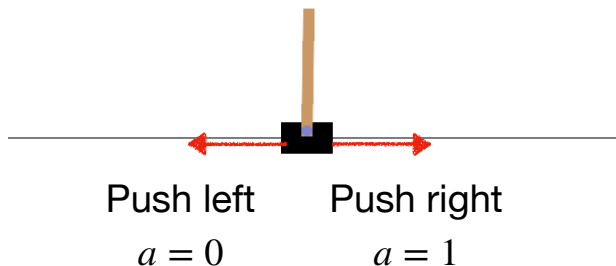
Deterministic vs stochastic?

Q: Assume S state and A actions, how many different deterministic policies we can have?

$$A \times A \times A \times A = A^4 \quad |A| \text{ states} \quad |S| \quad |A|^{|S|}$$

Example: OpenAI Gym demonstrations

State = [cart pos, cart velocity,
pole angle, pole angular velocity]

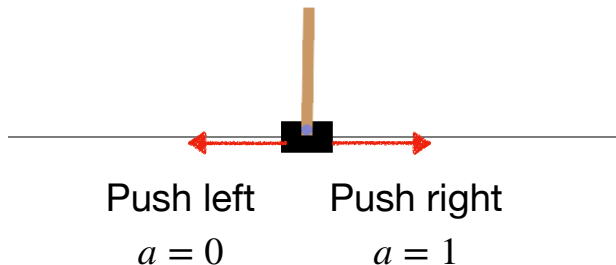


$$r(s, a) = \begin{cases} 1 & \text{pole angle} \in [-12^\circ, 12^\circ], \\ 0 & \text{else} \end{cases}$$

Policy 1: uniform random
 $\pi(0 | s) = \pi(1 | s) = 0.5, \forall s$
left *right*

Example: OpenAI Gym demonstrations

State = [cart pos, cart velocity,
pole angle, pole angular velocity]



$$r(s, a) = \begin{cases} 1 & \text{pole angle} \in [-12^\circ, 12^\circ], \\ 0 & \text{else} \end{cases}$$

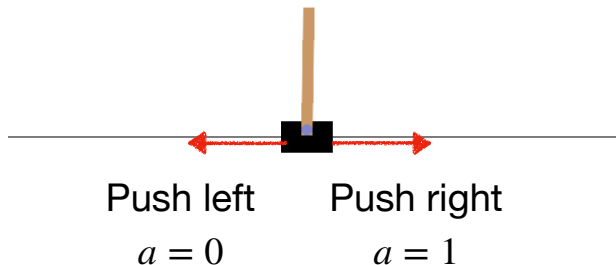
Policy 1: uniform random
 $\pi(0 | s) = \pi(1 | s) = 0.5, \forall s$

Policy 2: adaptive

$$\pi(s) = \begin{cases} 0(\text{left}) & \text{if pole angle} < 0 \\ 1(\text{right}) & \text{else} \end{cases}$$

Example: OpenAI Gym demonstrations

State = [cart pos, cart velocity,
pole angle, pole angular velocity]



$$r(s, a) = \begin{cases} 1 & \text{pole angle} \in [-12^\circ, 12^\circ], \\ 0 & \text{else} \end{cases}$$

Policy 1: uniform random
 $\pi(0 | s) = \pi(1 | s) = 0.5, \forall s$

Policy 2: adaptive

$$\pi(s) = \begin{cases} 0(\text{left}) & \text{if pole angle} < 0 \\ 1(\text{right}) & \text{else} \end{cases}$$

Outlines:



1. Definitions of Markov Decision Process

2. Value functions (V and Q functions)

3. Bellman equations

Performance of a policy

time step

Expected total reward of a policy π :

$\mathbb{E}_{\pi} [V(s)]$

$\mathbb{E}_{\pi} [V(s, \pi)]$

$\mathbb{E} [V(s, \pi)]$

$$\underline{V^{\pi}(s)} = \mathbb{E} \left[\underbrace{r_0}_{\text{D}} + \underbrace{\gamma r_1}_{\text{D}} + \underbrace{\gamma^2 r_2}_{\text{D}} + \dots + \underbrace{\gamma^h r_h}_{\text{D}} + \dots \mid s_0 = \underline{s}, \underline{\pi} \right]$$

$\gamma \in [0,1)$: discount factor (value future reward less and less)



Performance of a policy

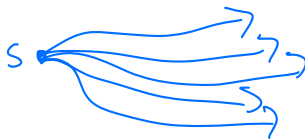
Expected total reward of a policy π :

$$V^\pi(s) = \mathbb{E} \left[r_0 + \gamma r_1 + \gamma^2 r_2 + \dots + \gamma^h r_h + \dots \mid s_0 = s, \pi \right]$$

$\gamma \in [0,1)$: discount factor (value future reward less and less)

$$\gamma^0 + \gamma^1 + \gamma^2 + \dots = \frac{1}{1-\gamma}$$

Q: think about the CartPole example, is there a way we can estimate $V^\pi(s)$ at a given s ?



Optimal policy

$\pi^\star$: the policy that maximizes expected future reward at all states

$$V^\star(s) \geq V^\pi(s), \forall s, \forall \pi$$

$$V^{\pi^\star} = V^\star$$

Optimal policy

$\pi^\star$: the policy that maximizes expected future reward at all states

$$V^\star(s) \geq V^\pi(s), \forall s, \forall \pi$$

Fact: $\pi^\star$ is deterministic, only depends on state (i.e., Markovian);
 $\pi^\star$ always exists for any infinite horizon discounted MDP